

Failure (a dead connection, and a real container)

— Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper error handling and a real Dockerfile), **Advanced** (the mistakes that actually leak secrets or crash containers in production). Read Basic first even if you've used Docker before — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

This exercise is really two separate things: breaking your database connection on purpose to prove your error handling actually works, and packaging your whole app into a container that runs the same way anywhere.

Things to use for the database part:

- Point your `sqlite3.connect(...)` call at a folder that doesn't exist, like `"/no/such/path/runs.db"`.
- Hit any route that touches the database, and check you get a `500` back — not a crash, not a raw Python traceback shown to the client.

Things to use for the Docker part:

- A `Dockerfile` with 4 lines: `FROM`, `COPY` + `RUN` (install), `COPY` (your code), `CMD` (how to start it).
- `docker build -t my-api .`
- `docker run -p 8000:8000 -e OPENAI_API_KEY=sk-... my-api`

Intermediate Version

A broken database connection should be caught by the same `try/except Exception` pattern `chat_route` already built — the difference here is proving it, on purpose, against something real (a genuinely bad path), not a fake exception you raise by hand.

For the `Dockerfile`, the *order* of the lines matters, not just their presence: `COPY requirements.txt .` and `RUN pip install -r requirements.txt` need to happen **before** `COPY . .` (your actual code). Docker caches each line as a layer; if `requirements.txt` hasn't changed, Docker reuses the cached install step instead of re-running `pip install` on every single build — which matters a lot once your dependency list is more than a couple of packages.

The exact pieces:

- `sqlite3.connect("/no/such/path/runs.db")` — this raises `sqlite3.OperationalError: unable to open database file` the moment you try to use it (SQLite doesn't create missing parent directories).

- A Dockerfile: `FROM python:3.11-slim WORKDIR /app COPY requirements.txt . RUN pip install -r requirements.txt COPY . . CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]`
- `--host 0.0.0.0` matters inside a container — `127.0.0.1` (uvicorn's default) only listens for connections from *inside* the container itself, so requests from outside (your `curl`, or `-p 8000:8000`) would never reach it.
- `docker run -e KEY=value` passes an environment variable in at run time — never baked into the image.

Advanced Version

Two things only show up once you actually run `docker history` and actually kill a connection mid-request, not just imagine doing it:

Secrets in image layers, for real: if you ever `COPY .env .` into the image, that `.env` file is now permanently part of one of the image's layers — even if a *later* `RUN rm .env` line seems to delete it, the earlier layer still has it, and `docker history --no-trunc my-api` (or `docker save + unpacking the tarball`) can reveal it. The only real fix is: never `COPY` a secret into the image in the first place. A `.dockerignore` file (listing `.env`, `.git/`, `__pycache__`, local venvs) is what stops this from happening by accident during `COPY . .`.

A dead connection mid-request, not just a bad path at startup: pointing at a nonexistent path fails the moment you try to use the connection — a slightly different, and arguably more realistic, failure is a connection that *was* working and then stops (the file gets deleted while the app is running, a network database's connection drops). Simulate this by closing the connection object yourself (`conn.close()`) from a separate script or a debug route, then immediately hitting a real route — you should still get a clean 500, proving your error handling isn't only correct for the "never worked" case.

Pieces:

- `.dockerignore` file: `.env`, `.git`, `__pycache__`, `*.pyc`, `.venv/`
- `docker history --no-trunc my-api` — inspect every layer for anything that shouldn't be there.
- A required env var that's genuinely missing at container startup should fail loudly and immediately, not hang — check this by reading `os.environ["OPENAI_API_KEY"]` (which raises `KeyError` immediately) rather than `os.getenv(...)` (which would return `None` and let the container start in a broken state) for anything the app truly cannot run without.

Difference between Basic, Intermediate, and Advanced: Basic names the pieces for both halves of this exercise — breaking a connection, and a minimal working Dockerfile. Intermediate explains exactly why the Dockerfile's line order matters for build speed, and why `--host 0.0.0.0` is required inside a container specifically. Advanced covers what only shows up under real inspection:

proving no secret survives in any image layer with `docker history` , and testing a connection that dies *mid-run* instead of one that was simply misconfigured from the start.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
temporarily change the db path to somewhere that doesn't exist
hit a route that touches the database
check: you get 500, not a crash, not a raw traceback

write a Dockerfile:
    start from python:3.11-slim
    copy requirements.txt, install it
    copy the rest of the code
    say how to start the app

docker build -t my-api .
docker run -p 8000:8000 -e OPENAI_API_KEY=sk-... my-api
curl localhost:8000/health # should work the same as running it locally
```

Here's almost the whole thing for the database half:

```
import sqlite3
from fastapi import FastAPI, HTTPException

app = FastAPI()
conn = sqlite3.connect("/no/such/path/runs.db", check_same_thread=False) # broke

@app.get("/runs")
def get_runs():
    try:
        rows = conn.execute("SELECT * FROM runs").fetchall()
        return rows
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")
```

Expected output (`curl -i localhost:8000/runs`):

```
HTTP/1.1 500 Internal Server Error
{"detail": "Something went wrong."}
```

And here's almost the whole Dockerfile:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Intermediate Version

same broken-connection route as Basic, but with a real try/except in the actual CREATE TABLE call too (not just the SELECT) – the open can fail before any query even runs

Dockerfile, with a .dockerignore alongside it:

```
.dockerignore excludes .env, .git, __pycache__, venvs
```

after building and running:

```
docker run -e OPENAI_API_KEY=sk-... -p 8000:8000 my-api
curl localhost:8000/health -> same output as running locally
docker history my-api -> confirm no secret value appears anywhere
```

```
# main.py
import sqlite3
from fastapi import FastAPI, HTTPException

app = FastAPI()

try:
    conn = sqlite3.connect("/no/such/path/runs.db", check_same_thread=False)
    conn.execute("CREATE TABLE IF NOT EXISTS runs (id INTEGER PRIMARY KEY, input
except sqlite3.OperationalError:
    conn = None # app still starts, but routes using it will fail cleanly

@app.get("/runs")
def get_runs():
    if conn is None:
        raise HTTPException(status_code=500, detail="Something went wrong.")
    try:
        return conn.execute("SELECT * FROM runs").fetchall()
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")
```

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# .dockerignore
.env
.git
__pycache__/*
*.pyc
.venv/
```

Expected output: `curl localhost:8000/runs` against the container returns `500` `{"detail": "Something went wrong."}` — the app starts fine even though its database path is broken, and fails cleanly, request by request, instead of crashing at startup.

Advanced Version

```
prove a *mid-run* death, not just a bad path from the start:
  start with a working database path
  hit /health once, confirm 200
  close the connection yourself (simulating it dying)
  hit a db-using route again, confirm a clean 500 – not a crash

prove no secret is in any image layer:
  docker history --no-trunc my-api
  grep the output for your real API key value – must find nothing

prove a missing required env var fails loudly at startup, not silently:
  docker run my-api # no -e OPENAI_API_KEY at all
  confirm: container exits immediately with a clear error, does not hang
```

Here's most of it — fill in the required-env-var check yourself:

```
# main.py
import os
import sqlite3
from fastapi import FastAPI, HTTPException

# your turn: read a required env var with os.environ["OPENAI_API_KEY"]
# (not os.getenv) so a missing key raises KeyError immediately at import time,
```

```
# crashing the container on startup with a clear traceback instead of starting
# in a silently broken state
...

app = FastAPI()
conn = sqlite3.connect("runs.db", check_same_thread=False)
conn.execute("CREATE TABLE IF NOT EXISTS runs (id INTEGER PRIMARY KEY, input TEXT

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/runs")
def get_runs():
    try:
        return conn.execute("SELECT * FROM runs").fetchall()
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")
```

```
# kill_connection.py – run this against the same process to simulate a mid-run de
# (in practice: a debug route, or just conn.close() in a REPL attached to the pro
conn.close()
```

Try it end to end: `docker run -e OPENAI_API_KEY=sk-... -p 8000:8000 my-api`, confirm `/health` works, then re-run with no `-e` flag at all and confirm the container exits immediately with a clear traceback naming the missing key — not a silent hang.

Difference between Basic, Intermediate, and Advanced: Basic covers the required cases once each — one broken path, one minimal Dockerfile. Intermediate handles the connection failing at *open* time (inside `try/except`, so the app still starts and fails per-request instead of crashing entirely) and adds a `.dockerignore` so secrets never enter a layer by accident. Advanced tests a connection that dies mid-run (not just one that never worked), proves with `docker history` that nothing leaked, and proves a genuinely missing required env var crashes the container immediately and loudly instead of starting in a broken, silent state.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)